

const, constexpr and the C const loophole

What C++ adds to the pointers, arrays and structures you just wrote in C — and what the compiler can check, or compute outright, before the program ever runs.

What this adds to Lab 3

PART 1

const

Why the lab's snippets are undefined behaviour in C, why the same lines do not compile in C++, and how to read a const declaration without guessing.

PART 2

constexpr

Values, functions, structures and whole lookup tables that the compiler evaluates before the program starts.

WORKED FROM

the lab

Snippets 1 to 4, the Tribonacci of Exercise 1, the triangle of Exercise 2, and the string exercises all come back as C++ examples.

Every example on the following slides compiles with **g++ -std=c++20 -Wall -Wextra**.

What C's const actually promises

LAB 3, CODE SNIPPET 1

```
const int i = 10;  
int *p = &i;  
(*p)++;  
printf("%d\n", i);
```

gcc says: **warning: initialization discards 'const' qualifier** — and produces an executable anyway.

A diagnostic, not a refusal

C requires the compiler to complain about the drop of the const qualifier. It does not require it to stop.

const qualifies the name

It is a property of the declaration, not a lock on the object. Another name for the same object can be non-const.

Writing through it is UB

C11 §6.7.3: modifying an object defined with a const-qualified type is undefined. Print 10, print 11, or fail, all are conforming.

Why it sometimes works: storage decides

DECLARATION	OBJECT LIVES IN	WRITE THROUGH A STRIPPED POINTER
<code>const int i = 10; // local</code>	the stack frame	usually succeeds — i really changes
<code>static const int i = 10;</code>	.rodata	page is read-only → SIGSEGV
<code>const char *s = "text";</code>	literal pool	s[0] = 'T' → SIGSEGV
<code>const int *cp = malloc(4);</code>	the heap	legal — the object was never const

The last row is the one worth remembering: **const** on a pointer describes the **view**, not the object. Casting it away is undefined when the object itself was declared const — rows one to three.

C++ turns the warning into an error

```
const int i = 10;
int* p = &i;           // error: invalid
                        // conversion
                        // from 'const int*' to
                        // 'int*'

int* q = const_cast<int*>(&i); // compiles – you asked
for it
*q = 11;               // still undefined
behaviour
```

No implicit conversion

In C++ there is no conversion from **const T*** to **T***. The line is ill-formed, full stop.

const_cast is the door

Loud, explicit, greppable. Casting away const is a decision somebody can point at in review.

The UB does not go away

`const_cast` removes the compiler's objection, not the rule. Its honest use is calling a C API that takes `int*` but does not write.

Reading a const declaration

DECLARATION	POINTER	POINTEE	SAY IT OUT LOUD
<code>const int* p;</code>	mutable	read-only	pointer to const int
<code>int const* p;</code>	mutable	read-only	exactly the same thing
<code>int* const p;</code>	fixed	mutable	const pointer to int
<code>const int* const p;</code>	fixed	read-only	const pointer to const int

Read right to left from the variable name. **const** binds to whatever sits immediately to its left — and only jumps right when there's nothing on the left, which is why the first two rows mean the same thing.

The lab's other snippets, re-read

SNIPPET 3 – const pointer to const

```
double c = 1.5;
const double* const p = &c;
c += 0.5;           // c itself is not const
// *p = c;          // error: pointee is const
// p = NULL;         // error: pointer is const
printf("%f\n", *p); // prints 2.000000
```

Both consts are real, and neither stops the write. The object is reachable under its own name, where nothing is const.

SNIPPET 2 – const pointer to non-const

```
char* const p = malloc(6);
memset(p, 'A', 6);
(*p)++;           // the chars are not const
// p = NULL;      // error: p cannot be
// reused
printf("%c\n", *p++); // error: p++ writes to
free(p);
```

The trap is on the printf line: ***p++** looks like a read, but the inc targets the pointer, which is the const half here.

Every one of these reduces to the same question: **which of the two things is const – the pointer, or what it points at?**

const in C++ carries more weight

GIVEN `CONST INT N = 5;`

C

C++

Is **n** a constant expression?

no — just a read-only variable

yes, for integral types

`int a[n];` // at file scope

error

an ordinary fixed-size array

`case n:` // in a switch

error

accepted

Linkage at file scope

external — duplicate symbol if a header defines it

internal — safe to put in a header

const on a function

no such thing

member functions can be const

Net effect: in C++, **const** does the job **#define** used to do in C — with a type, a scope and a value the debugger can see.

const member functions

THE LAB'S struct Person, IN C++

```
struct Person {
    std::string name;
    int         age;

    void show() const;           // this is const
Person*
    void birthday() { ++age; }  // may modify: not
const
};

void Person::show() const {
    std::printf("%s %d\n", name.c_str(), age);
    // ++age; error: member of a const object
}

const Person p{"Ada", 36};
p.show();           // OK
p.birthday();       // error: discards qualifiers
```

The object is the argument

A trailing **const** qualifies the hidden **this** parameter.
Only const member functions may be called on a const object.

mutable is the exception

A **mutable** member may still change inside a const member function — for caches, counters and mutexes.

const overloads

T& at(i) and **const T& at(i) const** coexist; the constness picks one.

const in function signatures

```
void show(const Person& p);    // reads it, copies nothing
void grow(Person& p);         // will modify the caller's
Person clone(Person p);       // copies, on purpose

size_t len(const char* s);    // promises not to write
void fill(char* s, char c);   // may write through s

void f(int x);
void f(const int x);  // error: redefinition of f(int)

void g(int* p);
void g(const int* p); // fine: two different functions
```

const& is the default

For anything bigger than a pointer, take a reference: no copy is made, and the header already says the argument is safe.

Documentation that is checked

A comment saying “does not modify” is far away from the code. A **const** in the signature cannot.

Top-level const is invisible

On a by-value parameter it affects only the body, so it does not overload. One level down on the pointee — it is part of the type.

Three common misreadings

“The bytes cannot change”

It means *this name* cannot change them. Another name for the same object is free to.

```
int x = 1;
const int& r = x;
x = 2;      // r reads 2
```

“const_cast makes it legal”

It makes it *compile*. If the object was declared const, the write is still undefined.

```
const int i = 10;
int* p = const_cast<int*>
(&i);
*p = 11;    // still UB
```

“const lives in read-only memory”

Only with static storage. A local const in an ordinary writable stack frame

```
static const int a = 1;
const int b = 1; // stack
```

There is a fourth one worth naming: the compiler is allowed to **assume** a const object never changes, so it may fold the old value code. A write that “worked” can still be invisible to the next read.

Two promises, two different questions

const

“May I write through this name?”

A promise about **access**. The value itself may be produced at run time, read from a file, or passed in as an argument.

constexpr

“Is this value known while compiling?”

A promise about **time**. On a variable it also implies **const** - cannot have one without the other.

```
const      int a = read_int();    // fine: known only at run time
constexpr int b = read_int();    // error: not a constant expression
constexpr int c = 3 * 7;         // 21, baked into the binary
int arr[c];                      // a genuine fixed-size array
```

Every **constexpr** variable is **const**. The reverse does not hold.

Exercise 1, evaluated at compile time

```
// Tribonacci: 0, 0, 1, 1, 2, 4, 7, 13, 24, 44, 81, ...

constexpr long long trib(int n) {
    long long a = 0, b = 0, c = 1;      // T0, T1, T2
    for (int i = 0; i < n; ++i) {
        long long t = a + b + c;
        a = b; b = c; c = t;
    }
    return a;
}

static_assert(trib(10) == 81); // checked by the
compiler

int table[trib(6)];           // an array of 7 ints

int n = read_int();

long long r = trib(n);        // same function, at run
time
```

A permission, not an order

A constexpr function **may** run at compile time. Given a run-time argument it is an ordinary function.

static_assert forces it

Used in a constant expression, the function **must** be evaluable there or the build fails — no fallback to run time.

Standards

Loops and local mutation in constexpr: C++17. Before that, a single return statement. More free static_assert: C++17.

What is allowed inside a constexpr function

Allowed

- Local variables and mutating them
- Loops, **if**, **switch**, recursion
- Calls to other constexpr functions
- Constructing and using literal types
- References and pointers to objects created during the same evaluation

Not allowed while evaluating

- Anything that would be undefined behaviour
- Calling a non-constexpr function
- Reading a global that is not itself constant
- reinterpret_cast**, and casts that lose constness
- Reading an uninitialised variable
- Memory allocated and not released in the same evaluation

C++11	one return statement, no loops, no local variables
C++14	loops, local variables, mutation — what the Tribonacci example needs
C++17	constexpr lambdas, if constexpr , inline constexpr variables
C++20	try/catch, virtual calls, allocation that is freed again, constexpr vector and string
C++23	goto , static locals, non-literal local variables, if constexpr

Exercise 2, without running the program

```
struct Point { double x, y; };    // a literal type

constexpr double area(Point a, Point b, Point c) {
    double s = a.x*b.y - b.x*a.y
              + b.x*c.y - c.x*b.y
              + c.x*a.y - a.x*c.y;
    return s < 0 ? -s / 2 : s / 2;
}

constexpr Point A{2.5, 6}, B{1, 2.2}, C{10, 6};
constexpr double S = area(A, B, C);    // 14.25, at build time

static_assert(S > 14.24 && S < 14.26);    // not an == test
```

Literal types

A plain aggregate like **Point** qualifies with extra work, so whole structures can live in constant expressions.

The binary holds the answer

No multiplication survives to run time. Disassemble it and you find the constant formula.

Still floating point

Compile-time doubles round exactly like ones, so **==** is no safer here than anywhere.

constexpr classes and template arguments

```
struct Buffer {
    char data[64]{};
    size_t n = 0;
    constexpr void push(char c) { data[n++] = c; }
    constexpr size_t size() const { return n; }
};

constexpr Buffer make() {
    Buffer b;
    for (char c : {'i', 'n', 'n', 'o'}) b.push(c);
    return b;
}

constexpr Buffer b = make();
static_assert(b.size() == 4);

inline constexpr size_t CAP = 8;    // safe in a header
std::array<int, CAP> ring{};         // a template argument
```

Part 2 · constexpr

Objects, not just numbers

A constexpr constructor makes the whole object usable at compile time. **b** is a fully built object living in the binary.

constexpr is not const

On a member function it says “can run at compile time”, nothing more. **push** modifies the object, so we write **const** separately when you mean it.

Where it becomes necessary

Template arguments, array bounds and **static_assert** accept constant expressions. This is the machinery behind **std::array**.

Strings and tables, built at compile time

```
constexpr std::string_view uni = "Innopolis";
static_assert(uni.size() == 9);
static_assert(uni[0] == 'I');
static_assert(uni.substr(0, 4) == "Inno");

constexpr auto table = [] { // C++17 constexpr lambda
    std::array<long long, 20> t{};
    t[2] = 1;
    for (int i = 3; i < 20; ++i)
        t[i] = t[i-1] + t[i-2] + t[i-3];
    return t;
}();

static_assert(table[10] == 81); // computed at build time
```

string_view is a const view

A pointer and a length, owning nothing. It replaces **const char*** plus **strlen**, and it can be used inside constant expressions.

Tables instead of magic numbers

The whole Tribonacci table is written into the code section. Nothing runs at start-up to build the table.

Where the lab meets this

The “inno” to “innopolis” string exercise is replaced by checks the build performs, instead of checks the length computed every run.

Which mode am I in, and what does it cost?

```
constexpr int parse(std::string_view s) {  
    if (std::is_constant_evaluated()) { // C++20  
        return simple_loop(s);  
    } else {  
        return vectorised(s);          // needs a real CPU  
    }  
}  
  
// if constexpr (std::is_constant_evaluated())  
//             always true: the condition itself  
//             is a constant expression
```

Two bodies, one function

The plain **if** is the point. Written as **if constexpr** is a well-known bug: the test is always taken.

Evaluation has a budget

Compilers cap the work: **-fconstexpr-ops-lim** in gcc, **-fconstexpr-steps** in clang. Exceeding is a compile error, not a fallback.

It is part of the interface

Callers may use it in a **static_assert** or an array bound. Removing **constexpr** later breaks the interface. Add it where it is genuinely wanted.

Work moved to compile time does not disappear — it moves into every build. That is usually a good trade, and occasionally a very

const vs constexpr, side by side

	const	constexpr
Means	I will not write through this name	the value is computed while compiling
Initialiser	any expression, run time included	must be a constant expression
Array size, case label, template argument	only integral types with a constant initialiser	always, by construction
Applies to	variables, pointers, member functions	variables, functions, constructors
Storage	a real object with an address	often none — folded into the instructions

Rule of thumb: reach for **constexpr** when the value could be known at compile time, and **const** everywhere else you are only reading

The rest of the family

constexpr C++20

Must run at compile time. Calling it with a run-time value is an error, not a fallback.

```
constexpr int sq(int x) { return x * x; }
int a = sq(5);    // 25
int b = sq(n);    // error: not constant
```

constinit C++20

Guarantees a static variable is initialised at compile time — no initialisation-order surprises. It is **not** const.

```
constinit int counter = trib(10);
++counter;    // OK: still writable
```

if constexpr C++17

The condition is decided while compiling and the branch not taken is never instantiated.

```
template <class T>
void print(T v) {
    if constexpr (std::is_same_v<T, int>)
        std::printf("%d\n", v);
    else
        std::printf("%f\n", v);
}
```

This is the type-safe replacement for Exercise 4's **sizeof** test: the type is known exactly, not guessed from a byte count.

What to take away

1. In C, **const** is a promise the compiler asks you to keep but does not enforce. Breaking it through a pointer is undefined behaviour, and it often looks like it worked.
2. C++ rejects the same code outright. **const_cast** is the only way through, and it leaves a mark that is easy to find.
3. **const** answers “may I write?”. **constexpr** answers “is it known at compile time?”. Every constexpr variable is const; the reverse is not true.
4. A **constexpr** function is a normal function that is also allowed to run during compilation. **static_assert** is how you prove it does.
5. In practice: take **const&** in signatures, mark a constant **constexpr** when its value could be known at build time, and let **static_assert** check what the compiler can already prove.

WHERE TO READ MORE

[cppreference — cv qualifiers](#)

[cppreference — constexpr](#)

[cppreference — static_assert](#)

[C++ Core Guidelines — section Con](#)